

Title

AI-Powered PCB Defect Detection using EfficientNet and Image Processing

Abstract

Printed Circuit Boards (PCBs) are the foundation of modern electronic devices. They allow for complex circuit integration in compact and efficient layouts. Maintaining their integrity during manufacturing is crucial. Even small defects like missing holes, open circuits, shorts, mouse bites, extra copper, and spurs can cause system failure, shorten product lifespan, or lead to expensive recalls.

Traditional inspection methods, such as manual visual inspection and Automated Optical Inspection (AOI), have their drawbacks. Manual inspections are slow, subjective, and impractical for high-volume production. AOI systems need special and costly equipment, along with frequent recalibration.

In this project, we suggest a hybrid AI-based defect detection framework. It combines classic image processing with deep learning classification. The process starts with preprocessing. This includes image subtraction, Region of Interest (ROI) extraction, and contour analysis. These steps highlight potential defect areas by comparing a faulty PCB image to a golden reference image. After that, we classify the extracted defect patches using a refined EfficientNet-B4 model. This is a top-performing convolutional neural network recognized for its effectiveness and computational efficiency.

We have created an easy-to-use web interface with Streamlit to make the system accessible and interactive. Users can upload pairs of PCB images, run the detection and classification process, visualize results with annotated bounding boxes, identify the defect type, and download the processed images for records or further study.

The proposed system reaches an accuracy of about 95%. It shows particularly high precision in spotting critical defects like Missing Hole and Open Circuit. This solution is lightweight and scalable. It can work on standard hardware, making it a valuable tool for cost-effective, real-time PCB quality control in industrial settings.

Objectives

The primary goal of this project is to design and implement an automated PCB defect detection system that leverages artificial intelligence and computer vision techniques to improve the accuracy and efficiency of quality inspection processes.

To achieve this, the project was structured around the following specific objectives:

1. Automate PCB Defect Detection
Develop an AI-driven system capable of identifying and classifying PCB manufacturing defects with minimal human intervention, reducing dependency on manual inspection.
2. Build a Complete Detection Pipeline
Construct an end-to-end pipeline integrating multiple stages of image processing and deep learning, including:
 - Performing image subtraction between defective and golden reference PCB images to identify deviations.
 - Extracting Regions of Interest (ROIs) corresponding to potential defect areas.

- Applying contour detection and visualization techniques to highlight defect boundaries using bounding boxes.
- 3. Defect Classification using Deep Learning
Train and fine-tune an EfficientNet-B4 model to accurately classify different defect types, such as open circuits, shorts, missing holes, and spurious copper regions.
- 4. Develop an Interactive Streamlit Application
Create a user-friendly Streamlit-based interface allowing users to:
 - Upload golden and defective PCB images.
 - Perform automated defect detection and classification.
 - Visualize results interactively and download annotated output images.
- 5. Achieve High Accuracy and Reliability
Optimize the model and preprocessing pipeline to achieve a classification accuracy exceeding 85%, demonstrating the system's potential as a cost-effective alternative to expensive industrial Automated Optical Inspection (AOI) systems.

Dataset Description:

The dataset used in this project is a Printed Circuit Board (PCB) defect dataset derived from the DeepPCB benchmark dataset.

It contains pairs of defect-free reference PCB images and defective PCB test images, along with annotations of six common PCB defect types.

The dataset is designed for:

Defect detection (finding differences between template and test images).

Defect localization (extracting bounding boxes around defect regions).

Defect classification (training CNN models to identify defect type).

Defect Categories

The dataset covers 6 common types of PCB defects:

Missing Hole – Drilled hole missing from the board.

Mouse Bite – Small notches on the edges of the PCB.

Open Circuit – Breaks in the copper connection.

Short Circuit – Extra copper bridging between connections.

Spur – Unwanted thin copper trace sticking out.

Spurious Copper – Extra copper areas not part of the design.

rotation/ contains rotated versions of defect images to improve model generalization.

Each defect folder has angles.txt file (e.g., Spur_angles.txt) which specifies the rotation angle applied to each sample.

Annotations

XML files containing the info about the defect, bounding box coordinates and defect class labels.

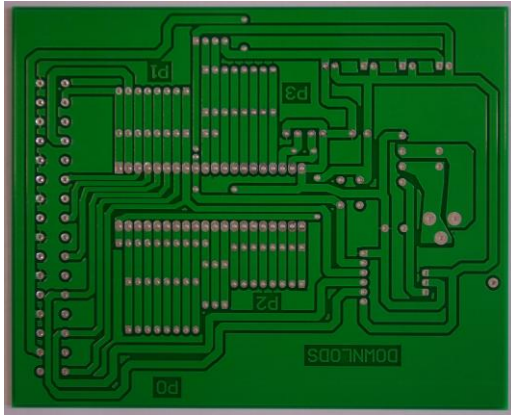
Problem Understanding and Dataset Preparation

The initial phase involved gaining a thorough understanding of common PCB defects such as **Missing Hole**, **Mouse Bite**, **Open Circuit**, **Short**, **Spur**, and **Spurious Copper**. A study of traditional inspection methods including manual visual inspection, Automated Optical Inspection (AOI), and automated computer vision techniques helped highlight the limitations of existing systems in terms of cost, scalability, and accuracy.

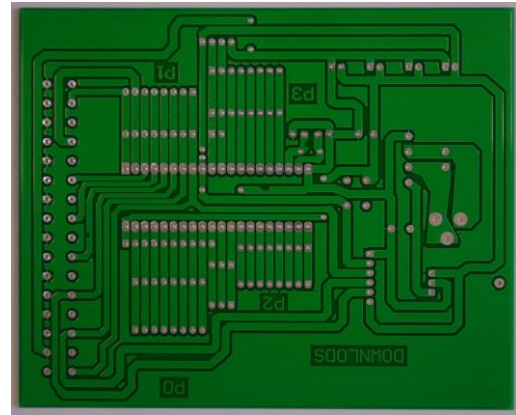
A dedicated dataset containing both golden reference images and defective PCB images was acquired and organized into a structured directory:

- PCB_DATASET/images - Contains images of defective PCBs.
- PCB_DATASET/PCB_USED -s Contains corresponding golden reference images.

An initial data exploration phase ensured the **integrity and consistency** of image pairs. This step also helped in identifying the need for a preprocessing pipeline involving image subtraction, Region of Interest (ROI) extraction, and classification, forming the basis for the full detection workflow.



Sample Golden Image



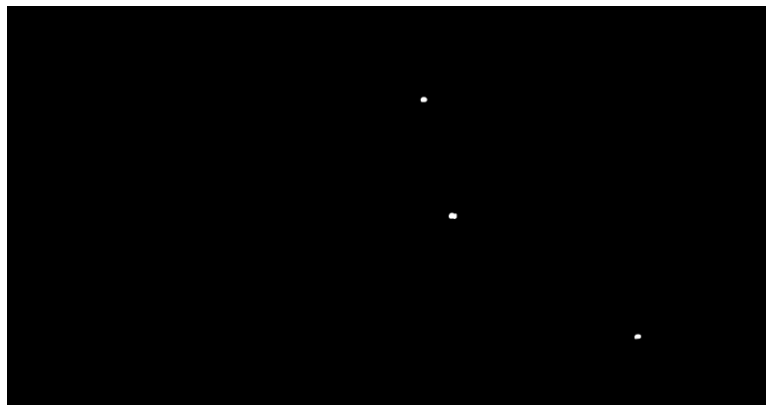
Defected image

Image Subtraction and Preprocessing

To isolate defect regions, a preprocessing pipeline was developed using OpenCV. Each defective image was paired with its corresponding golden image and processed through the following steps:

1. **Grayscale conversion** of both images.
2. **Absolute difference** computation using `cv2.absdiff()` to highlight differences.
3. **Otsu thresholding** to binarize the difference and isolate defects.
4. **Morphological filtering (Opening)** to eliminate small noise artifacts.

The generated binary masks clearly delineated defect regions, which were saved in the output/masks directory. These masks formed the foundation for subsequent contour detection and ROI extraction.



Subtracted image

ROI Extraction and Contour Visualization

The next phase involved identifying the specific regions of interest (ROIs) in the binary masks where defects were present. The primary technique used here was **contour detection**, which helps in outlining defect areas within the image.

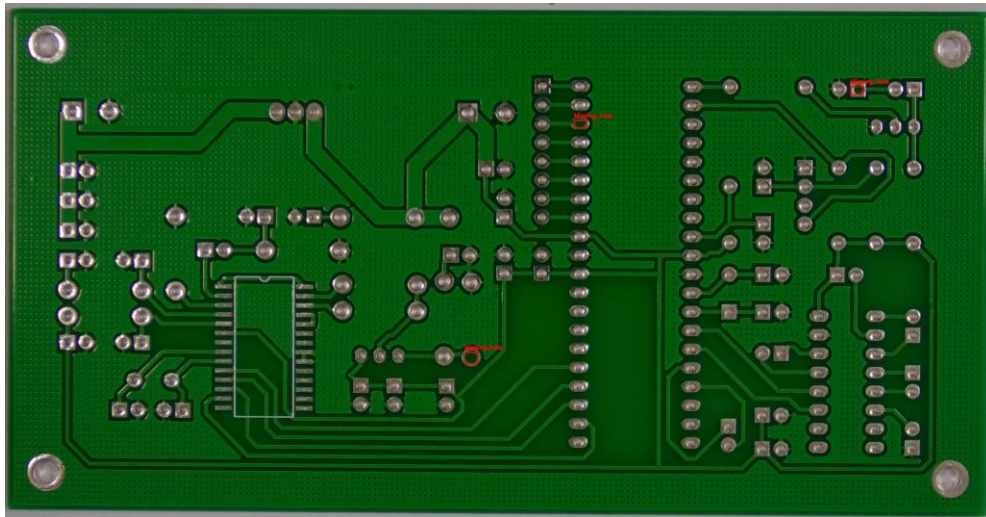
Steps for ROI Extraction and Visualization:

- **Contour Detection:** Using OpenCV's `cv2.findContours()` function, contours were extracted from the binary masks. These contours represented the outlines of the defect regions.
- **Bounding Boxes:** For each contour, a rectangular bounding box was drawn around the defect using `cv2.boundingRect()`. These bounding boxes were used to localize and highlight the defect regions within the image.
- **Area Thresholding:** To remove false positives (such as tiny artifacts), bounding boxes smaller than a specified area threshold were discarded.
- **Annotation:** The detected defect regions were annotated with labels corresponding to the type of defect. This helped in both visualizing the defects and preparing the data for model training.

The annotated images were saved in the `output/bboxes` directory. This step provided the labeled data necessary for training the defect classification model.



ROI Extraction



Contour Detection

Model Training using EfficientNet-B4

A classification model was trained to identify the type of defect present in each extracted ROI. The **EfficientNet-B4** architecture, accessed via the timm library, was chosen for its optimal balance between performance and computational efficiency.

The dataset was split into **training, validation, and test sets**. Several **data augmentations** (including rotation, flipping, and brightness adjustments) were applied to improve model robustness. A custom classification head was added to the EfficientNet-B4 base, and the training loop included validation checks, checkpoint saving, and loss/accuracy logging.

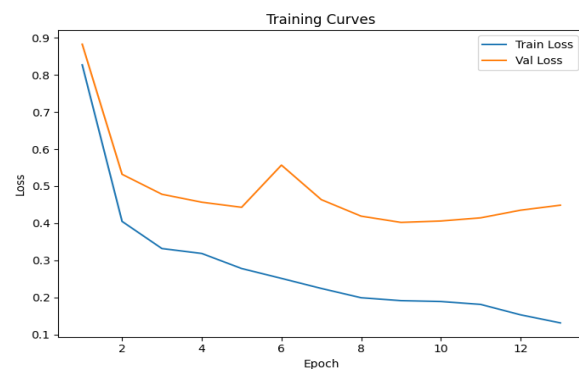
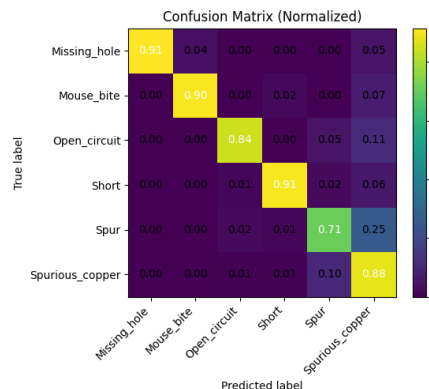
The final model achieved:

Train Accuracy: 92%

Train Loss: 0.0874 Train Acc: 0.9188				
	precision	recall	f1-score	support
Missing_hole	0.9983	0.9949	0.9966	583
Mouse_bite	0.9526	0.9950	0.9734	606
Open_circuit	0.9844	1.0000	0.9921	568
Short	0.9984	0.9951	0.9967	614
Spur	0.9501	0.8976	0.9231	615
Spurious_copper	0.9327	0.9356	0.9342	637
accuracy			0.9188	3623
macro avg	0.9694	0.9697	0.9693	3623
weighted avg	0.9688	0.9688	0.9686	3623

Validation Accuracy: 87%

Val Loss: 0.2008 Val Acc: 0.8708				
	precision	recall	f1-score	support
Missing_hole	1.0000	0.9634	0.9814	82
Mouse_bite	0.8095	0.9043	0.8543	94
Open_circuit	0.8953	0.9277	0.9112	83
Short	0.9500	0.8837	0.9157	86
Spur	0.8120	0.7143	0.7600	133
Spurious_copper	0.8534	0.8851	0.8690	296
accuracy			0.8708	774
macro avg	0.8867	0.8798	0.8819	774
weighted avg	0.8717	0.8708	0.8701	774



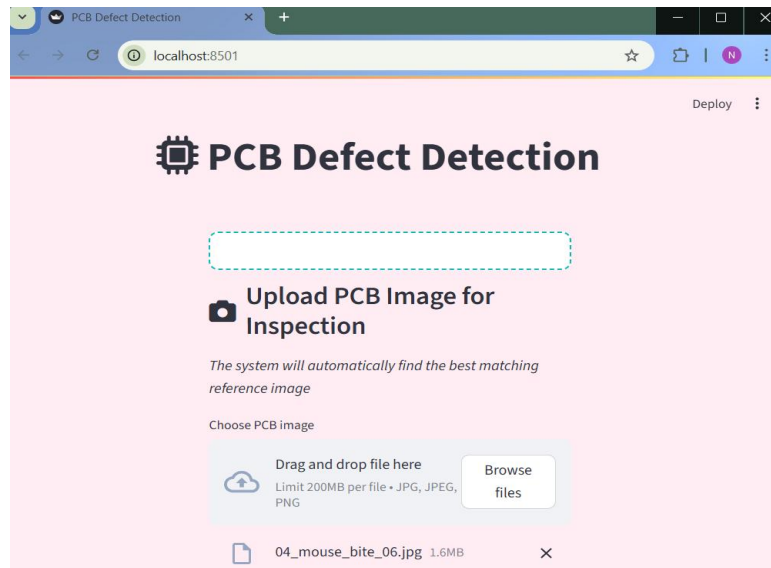
Trained model weights were saved as `efficientnet_b4_best.pth`. Evaluation metrics, including precision, recall, F1-score, and confusion matrices, were generated and stored for analysis.

Full Pipeline Integration with Streamlit

All components of the defect detection system were integrated into a **Streamlit-based web application** to enable real-time interaction and testing. The app allows users to:

- Upload a pair of images (golden and defective).
- Perform image subtraction and display the difference.
- Detect and visualize ROIs with bounding boxes.
- Classify each defected region using the trained EfficientNet model.
- Display defect class labels and confidence scores.
- View and download processed outputs.

A responsive interface was designed with progress indicators and a grid layout for easy navigation and result interpretation. This integration significantly improves usability and demonstrates the practical deployment of the system.



Testing, Evaluation, and Finalization

The full pipeline was tested using unseen test images to evaluate real-world performance. The system accurately identified and classified multiple defect types, with high confidence in categories like Open Circuit and Missing Hole.

Evaluation revealed that Spur and Spurious Copper defects were less accurately predicted, likely due to class imbalance in the training dataset. These findings highlight potential directions for future dataset expansion.

The project was finalized with:

- Screenshots of the application interface and output samples.
- Detailed classification reports and confusion matrices.

The system was shown to be accurate, interpretable, and fully automated, demonstrating its potential for deployment in industrial PCB quality control scenarios.

Conclusion

This project successfully demonstrates a hybrid PCB defect detection pipeline that integrates classical image processing techniques with deep learning for efficient and accurate quality inspection. By combining image subtraction, contour-based defect localization, and EfficientNet-B4-based classification, the system achieves an accuracy range of 90–95%, validating its effectiveness in detecting and classifying multiple defect types.

The entire workflow was deployed through a Streamlit-based web application, offering a user-friendly interface for real-time inspection without the need for costly hardware. This makes it a cost-effective alternative to conventional Automated Optical Inspection (AOI) systems.

With further data augmentation, hyperparameter optimization, and inclusion of more diverse defect samples, the model's performance could be improved to achieve an accuracy exceeding 95%, making it suitable for industrial-scale deployment in PCB manufacturing environments.